First-Order Logic

Going Beyond Propositional Logic I

We explored Propositional Logic (PL). It allowed us to represent and reason about facts that are either true or false. It is powerful for modeling simple boolean scenarios (e.g., whether a light is on or off or whether a person is present or absent), but what if we want to express more nuanced ideas:

- Every student in the class submitted the assignment.

  PL lacks ***quantifiers***: 'every'. We would need a separate proposition for each student (e.g., `Submitted_Alice`, `Submitted_Bob`). This does not scale or generalize.

- Some professors teach multiple courses.

  PL cannot represent ***relationships*** between objects (e.g., professors and courses). We need a predicate like `Teaches(ProfX, CourseY)`.

- If a person is a student, then they have an advisor.

  PL cannot handle ***variables*** or ***object properties***. We would need a rule like `Student(x)` ⇒ `HasAdvisor(x)`.

First-Order Logic

First-Order Logic (FOL) is a formal system for representing knowledge and reasoning about objects, their properties, and relationships between them.

Properties

- More expressive than propositional logic
- Uses variables, predicates, and quantifiers

Capabilities

- Foundation for knowledge representation in AI
- Enables reasoning about infinite domains

First-Order Logic is sometimes refered to as *Predicate Logic* or *Predicate Calculus*.

Syntax: Terms

Definition. A ***term*** represents an object in the domain.

Types	Examples
Constants: specific objects	John, 42, Alice, 2, Madison, Green
Variables: placeholders for objects	$x, y, person, number$
Functions: computed objects (map objects to objects)	Father(John) — The father of John Plus(2,3) — The result of 2 + 3 LeftLeg(x) — The left leg of x

Definition. A ***ground term*** is a term with no variables.

Terms vs Sentences

Terms

Refer to objects in the domain:

`John`	a specific person
`42`	a specific number
`Father(John)`	John's father (a person)
x	any object (placeholder)

Terms are not true or false, they just name objects

Sentences

Sentences make claims that can be evaluated as true / false.

`Student(Alice)`	Is Alice a student?
`Tall(John)`	Is John tall?
`Loves(Jesse, Jordan)`	Does Jesse love Jordan?
$\forall x$ `Student(`x`)` $\Rightarrow$ `HasLaptop(`x`)`	Does every student have a laptop?

Sentences are true / false: they make assertions.

`Father(John)` refers to someone.

`Tall(Father(John))` claims something.

Distinguishing Functions from Predicates

In FOL, predicates and functions use the same syntax (`Symbol(arg₁, arg₂, ...)`), creating potential ambiguity: `Father(John)` might be a function (term) or predicate (sentence) depending on how `Father` is defined, whereas `IsFather(John)` is clearly a predicate.

Context and declaration are needed to disambiguate.

Function	Predicate
`Father(John)` Returns John's father.	`IsFather(John)` Is John a father?
`Tall(Father(John))` Is the father of John tall?	`IsFather(John)` ∧ `Doctor(John)` Compound claim: Is John a father and is John a doctor?
Note, functions appear inside predicates: `Tall(Father(John))`	

Is `Father(John)` ∧ `Doctor(John)` a valid compound sentence?

No. `Father(John)` is not a sentence.

You might write: $\exists x\ (x = \texttt{Father(John)} \wedge \texttt{Doctor}(x))$

Syntax: Atomic Sentences

Intuitive Definition. An ***atomic sentence*** is the simplest sentence in FOL:

$$P(t_1, t_2, \ldots, t_n)$$

P is a predicate symbol and each t_i is a term.

Atomic sentences make a single assertion without logical connectives (∧, ∨, ¬, ⇒) or quantifiers (∀, ∃), and can be evaluated as true or false.

Why can P not be a function?

Arity refers to the number of arguments a predicate takes.		
Unary	`Tall(John)`	Property of an object
Binary	`Loves(Alice, Bob)` -	Relation between two objects
Ternary	`Between(Madison, Chicago, Milwaukee)`	Relation among three objects
***n*-ary**	`Grade(Alice, CSC-343, Fall, 2026, A, …)`	In general, predicates can take any number of arguments

Syntax: Complex Sentences (Quantifiers)

Logical Connectives (same interpretation as with propositional logic)

Not: $\sim\phi$

And: $\phi \wedge \psi$

Or: $\phi \vee \psi$

Implies: $\phi \Rightarrow \psi$

Iff: $\phi \Leftrightarrow \psi$

Quantified Statements

Universal ($\forall$): Statement holds for ***all*** objects

$\forall x\ \mathrm{Student}(x) \Rightarrow \mathrm{HasLaptop}(x)$

All students have a laptop.

$\forall x\ \mathrm{CS_Major}(x) \Rightarrow \mathrm{Take}(x, CSC_223)$

All CS majors must take CSC-223.

Existential ($\exists$): Statement holds for ***at least one*** object

$\exists x\ \mathrm{Student}(x) \wedge \mathrm{GPA}(x, 4.0)$

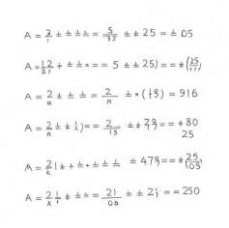
Some student has a 4.0 GPA

$\exists x\ \mathrm{Student}(x) \wedge \mathrm{Equals}(\mathrm{GPA}(x), 4.0)$

Some student has a 4.0 GPA

$\exists x\ \mathrm{Student}(\mathrm{John}, x) \wedge \mathrm{Doctor}(x)$

John is a student of someone who is a doctor.



Syntax: Mixed Quantifiers

Order matters!

We assume a domain where x ranges over students, y ranges over courses.

$$\forall x\ \exists y\ \text{Enrolled}(x, y)$$

For each x, we can choose a different y

For all students…

Every student is enrolled in some course.

(Each student can be in a different course.)

$$\exists y\ \forall x\ \text{Enrolled}(x, y)$$

This same y must work for all x

There exists a course…

There's a course that every student is enrolled in.

(One specific course contains all students.)

Syntax: Mixed Quantifiers II

Read mixed quantifiers left to right, thinking about dependency:

$\forall x\ \exists y$ "for each x, find a y"
(y depends on x)

The y can be different for each x.

$\exists y\ \forall x$ "find one y that works for all x"

The y must be the same for all x.

$$\forall x\ \exists y\ Loves(x, y)$$

"Everyone loves someone"

(Each person loves at least one person; could be different people.)

$$\exists y\ \forall x\ Loves(x, y)$$

"Someone is loved by everyone"

(There is one specific person that everyone loves.)

Question

Translate each FOL sentence to natural English; **Domain: People**.

	English Description	Interpretation
$\forall x\ \exists y\ Likes(x, y)$	Everyone likes someone	Each person likes at least one person, possibly different people
$\exists y\ \forall x\ Likes(x, y)$	There's someone everyone likes	One specific person is liked by everyone
$\forall x\ \exists y\ Knows(y, x)$	Everyone is known by someone	Each person has at least one person who knows them
$\exists x\ \forall y\ Knows(x, y)$	There's someone who knows everyone	One person knows all people
$\forall x\ \forall y\ Likes(x, y)$	Everyone likes everyone	All people like all people
$\exists x\ \exists y\ Likes(x, y)$	Someone likes someone	At least one person likes at least one person

Assume we read left to right:

$Likes(x, y)$ means "x likes y"

Question

Consider the following domain:

- People: {Alice, Bob, Carol}

- $Knows(Alice, Bob) = true$
- $Knows(Bob, Carol) = true$
- $Knows(Carol, Alice) = true$

- (All other $Knows$ relationships are false.)

Evaluate each statement as true or false.

Assume we read left to right:
$Knows(x, y)$ means "x knows y"

$\forall x\ \exists y\ Knows(x, y)$ *True*

Every person knows at least one other person.

$\exists y\ \forall x\ Knows(x, y)$ *False*

There is a single person that everyone knows.

$\forall x\ \exists y\ Knows(y, x)$ *True*

Everyone is known by someone.

$\exists x\ \forall y\ Knows(x, y)$ *False*

There is a person who knows everyone.

Question

Each sentence below has an error in the FOL translation. Identify and fix it:

Every student has an advisor who advises them.

$$\exists x\ Student(x) \Rightarrow \exists y\ Advisor(y)\ \wedge\ Advises(y, x)$$

Uses $\exists x$ instead of $\forall x$.

Every professor teaches at least one course.

$$\forall x\ \exists y\ Professor(x)\ \wedge\ Course(y) \Rightarrow Teaches(x, y)$$

This says, "For all x, there exists y such that:
if (x is a professor AND y is a course), **then** x teaches y."

Implications are of the form

```
If [condition], then [consequence]
```

Correct Form:

$$\forall x\ Professor(x) \Rightarrow \exists y\ Course(y)\ \wedge\ Teaches(x, y)$$

Question

How many entries does a truth table have for a FOL sentence with k variables where each variable can take on n values?

A. Truth tables are not applicable to FOL

B. 2^k

C. n^k

D. It depends

A. Truth tables do not work for FOL because the domain can be infinite, and we need to consider all possible interpretations of predicates and functions, not just variable assignments.

Quantifiers: Important Patterns

ϕ	ψ	$(\phi \rightarrow \psi)$	$(\sim\phi \mid \psi)$
T	T	T	T
T	F	F	F
F	T	T	T
F	F	T	T

Universal (∀) typically uses implication (⇒)

$\forall x\ Student(x) \Rightarrow Smart(x)$

"All students are smart"

$\forall x\ Student(x) \wedge Smart(x)$

"Everything is both a student and smart"

Existential (∃) typically uses conjunction (∧)

$\exists x\ Student(x) \wedge Smart(x)$

"Some student is smart"

$\exists x\ Student(x) \Rightarrow Smart(x)$

True if anything is not a student!

$\equiv \exists x(\sim Student(x) \vee Smart(x))$

Question

Translate to FOL.

1. "All birds can fly"
2. "Some students have laptops"
3. "No one is perfect"
4. "Everyone likes ice cream or cake"
5. "If someone is a teacher, then they have students"

1. $\forall x\ Bird(x) \Rightarrow CanFly(x)$
2. $\exists x\ Student(x) \wedge HasLaptop(x)$
3. $\neg\exists x\ Perfect(x)\ \ or\ \ \forall x\ \neg Perfect(x)$
4. $\forall x\ Likes(x, IceCream) \vee Likes(x, Cake)$
5. $\forall x\ Teacher(x) \Rightarrow \exists y\ Student(y) \wedge Teaches(x, y)$

Symmetry and Transitivity

"If x is a sibling of y, then y is a sibling of x"

$$\forall x\ \forall y\ Sibling(x, y)\ \Rightarrow\ Sibling(y, x)$$

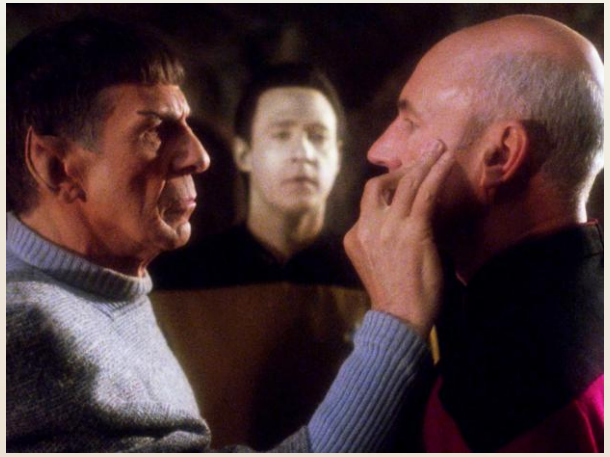
This states that the sibling relation is ***symmetric***.

"If x is an ancestor of y, and y is an ancestor of z, then x is an ancestor of z"

$$\forall x\ \forall y\ \forall z\ (Ancestor(x, y)\ \wedge\ Ancestor(y, z))\ \Rightarrow\ Ancestor(x, z)$$

This states that the ancestor relation is ***transitive***.

Unification



Unification: Matching Rules to Facts

We have facts in our knowledge base: $Student(Alice)$
$Takes(Alice, CSC343)$

We have general rules: $\forall x\ (Student(x)\ \wedge Takes(x, CSC343)) \Rightarrow Smart(x)$

Can we conclude $Smart(Alice)$?

Unification: Matching Rules to Facts II

The challenge is that variables in rules can match *any* term

Which of these could match $Student(x)$?

- ✓ $Student(Alice)$
- ✓ $Student(Bob)$
- ✓ $Student(FatherOf(Charlie))$
- ✗ $Teacher(Alice)$ — Different predicate
- ✗ $Student(x, \text{CSC_343})$ — Different arity

We need a systematic way to:

1. Find what value(s) make expressions match
2. Apply those values consistently throughout reasoning
3. Handle complex nested terms

Unification to the rescue!

Unification: The Main Idea

Unification finds substitutions that make expressions identical.

Rules	$Likes(Alice, x)$
Facts	$Likes(Alice, Pizza)$
What should x be?	$x = Pizza$

Goal: Make two expressions match by finding substitutions for variables.

Unification enables automated reasoning by systematically matching facts with rules so we can apply inference in FOL and power systems like Prolog.

Once we can unify, we can automate logical reasoning!

Unification Examples

Example: Multiple variables

```
Sibling(x, y)
Sibling(John, Mary)

Unifier: { x/John, y/Mary }
```

Example: Variables on both sides

```
Parent(x, Bob)
Parent(Alice, y)

Unifier: {x/Alice, y/Bob}
```

Example: Nested Terms

```
Ancestor(FatherOf(x), y)
Ancestor(FatherOf(John), Mary)

Unifier: { x/John, y/Mary }
```

Example: Functions Unifying with Variables

```
Age(x)
Age(MotherOf(Alice))

Unifier: { x/ MotherOf(Alice)) }
```

When Unification Fails

Example: Different Constants

`Student(Alice)`

`Student(Bob)`

Cannot Unify: `Alice` ≠ `Bob`

Example:

`Student(x)`

`Teacher(x)`

Cannot Unify: Different Predicates

Example: Occurs Check

`x`

`f(x)`

Cannot unify! `x` appears inside `f(x)`.

The occurs check prevents infinite substitutions.

If we unify `x` with `f(x)` to acquire the substitution `{x / f(x)}` (that is, `x <- f(x)`), then whenever we need to use the value of `x`, we replace it with `f(x)`.

```
x <- f(x)
x <- f(f(x))
x <- f(f(f(x)))
```

Unification Algorithm

```
function UNIFY(exp1, exp2, Θ):          # Θ is a list of substitutions
    if Θ = FAIL then return FAIL        # If already failed, propagate failure
    if exp1 = exp2 then return Θ        # Identical expressions need no substitutions

    if exp1 is a variable then
        return UNIFY-VAR(exp1, exp2, Θ)
    else if exp2 is a variable then
        return UNIFY-VAR(exp2, exp1, Θ)

    if exp1 and exp2 are functions/predicates then
        if they have same name and arity then
            UNIFY-ARGS(args(exp1), args(exp2), θ)
        else return FAIL    # Different functions or arity

    return FAIL     # Cannot unify: no other cases match
```

{x/Alice, y/Bob}

Likes(x, y) = Likes(x, y)

Handle either expression being variables

x , Alice x , Father(Bob)

Alice , x Father(Bob) , x

Handle compound expressions

Knows(x, y) and Knows(Alice, Bob)

Unify all arguments pairwise

{x/Alice, y/Bob}

UNIFY-VAR

```
function UNIFY-VAR(var, x, Θ):
    # If var already has a binding, unify its value with x
    if Θ(var) = val then
        return UNIFY(val, x, Θ)

    # If x is a variable with a binding, unify var with x's value
    else if x is a variable and Θ(x) = val then
        return UNIFY(var, val, Θ)

    # Prevent binding var to a term containing var (prevent
    # infinite structures)
    else if OCCURS-CHECK(var, x) then return FAIL

    # No conflicts: add the new binding Θ(var) = x
    else return Θ ∪ {var / x}
```

var: z
x : Alice → UNIFY(Bob, Alice, Θ) → FAIL
Θ: {z/Bob}

var: z
x: w → UNIFY(z, Alice, Θ) → {w/Alice, z/Alice}
Θ = {w/Alice}

var: z
x: f(z) → FAIL
Θ = {}

var: z
x: Alice → Θ = {z/Alice}
Θ = {}

Goal: Given a variable and an expression, find a substitution that makes them identical, following any existing bindings and preventing circular references.

UNIFY-VAR Examples

```
function UNIFY-VAR(var, x, Θ):
    if Θ(var) = val then
        return UNIFY(val, x, Θ)
    else if x is a variable and Θ(x) = val then
        return UNIFY(var, val, Θ)
    else if OCCURS-CHECK(var, x) then return FAIL
    else return Θ ∪ {var / x}
```

Simple variable binding	UNIFY-VAR(x, Alice, {})	→	{x/Alice}
Following existing bindings	UNIFY-VAR(y, x, {x/Alice})	→	{x/Alice, y/Alice}
Variable already bound	UNIFY-VAR(x, Bob, {x/Alice})	→	FAIL
Binding to a complex term	UNIFY-VAR(z, MotherOf(Alice), {})	→	{z/MotherOf(Alice)}
Chain of variable bindings	UNIFY-VAR(z, y, {x/Alice, y/x})	→	{x/Alice, y/x, z/Alice}
Occurs check violation	UNIFY-VAR(x, Parent(y, x), {y/Alice})	→	FAIL

UNIFY-ARGS

```
function UNIFY-ARGS(args1, args2, Θ):
    # Base case: no more arguments to unify
    if args1 and args2 are both empty then return Θ

    # Different number of arguments - should never
    # happen; checked in UNIFY
    if args1 = ∅ and args2 ≠ ∅ then return FAIL
    if args1 ≠ ∅ and args2 = ∅ then return FAIL

    # Unify the first pair of arguments from the
    # 'parallel' lists
    Θ' = UNIFY(head(args1), head(args2), Θ)

    # If that failed, propagate failure
    if Θ' = FAIL then return FAIL

    # Recursively unify remaining arguments with
    # updated substitution
    return UNIFY-ARGS(tail(args1), tail(args2), Θ')
```

Goal: Given two parallel lists, unify arguments one at a time (left to right), threading the substitution through each step.

$$L = [\textbf{head} \mid \textbf{tail}]$$
$$\equiv L[0] + L[1:]$$

UNIFY-ARGS Examples I

```
function UNIFY-ARGS(args1, args2, Θ):
    if args1 and args2 are both empty then return Θ

    if args1 = ∅ and args2 ≠ ∅ then return FAIL
    if args1 ≠ ∅ and args2 = ∅ then return FAIL

    Θ' = UNIFY(head(args1), head(args2), Θ)

    if Θ' = FAIL then return FAIL

    return UNIFY-ARGS(tail(args1), tail(args2), Θ')
```

UNIFY-ARGS([Alice, x], [y, Bob], {})

UNIFY(Alice, y, {}) → {y/Alice}

UNIFY-ARGS([x], [Bob], {y/Alice})

UNIFY(x, Bob, {y/Alice}) → {y/Alice, x/Bob}

UNIFY-ARGS([x, x], [Alice, y], {})

UNIFY(x, Alice, {}) → {x/Alice}

UNIFY-ARGS([x], [y], {x/Alice})

UNIFY(x, y, {x/Alice}) → {x/Alice, y/Alice}

UNIFY-ARGS Examples II

```
function UNIFY-ARGS(args1, args2, Θ):
    if args1 and args2 are both empty then return Θ

    if args1 = ∅ and args2 ≠ ∅ then return FAIL
    if args1 ≠ ∅ and args2 = ∅ then return FAIL

    Θ' = UNIFY(head(args1), head(args2), Θ)

    if Θ' = FAIL then return FAIL

    return UNIFY-ARGS(tail(args1), tail(args2), Θ')
```

```
UNIFY-ARGS([Alice, x], [Bob, y], {})

          UNIFY(Alice, Bob, {})      →    FAIL
```

```
UNIFY-ARGS([x, MotherOf(y)],
           [Alice, MotherOf(Bob)], {})

          UNIFY(x, Alice, {}) → {x/Alice}

          UNIFY-ARGS([MotherOf(y)], [MotherOf(Bob)], {x/Alice})

          UNIFY(MotherOf(y), MotherOf(Bob), {x/Alice})

                UNIFY-ARGS([y], [Bob], {x/Alice})

                UNIFY(y, Bob, {x/Alice})      →    {y/Alice, x/Bob}
```

UNIFY Example

```
UNIFY(Parent(John, MotherOf(x)),
      Parent(y, MotherOf(Bob)), {})
```

Same predicate and arity

```
UNIFY-ARGS([John, MotherOf(x)],
           [y, MotherOf(Bob)], {})
```

```
UNIFY(John, y, {}) → {y/John}

UNIFY(MotherOf(x), MotherOf(Bob), {y/John})

UNIFY-ARGS([x], [Bob], {y/John})

UNIFY(x, Bob, {y/John})      →    {y/John, x/Bob}
```

```
function UNIFY(exp1, exp2, Θ):
    if Θ = FAIL then return FAIL
    if exp1 = exp2 then return Θ
    if exp1 is a variable then
        return UNIFY-VAR(exp1, exp2, Θ)
    else if exp2 is a variable then
        return UNIFY-VAR(exp2, exp1, Θ)
    if exp1 and exp2 are functions/predicates then
        if they have same name and arity then
            UNIFY-ARGS(args(exp1), args(exp2), θ)
        else return FAIL
    return FAIL
```

Inference in First-Order Logic

Toward Inference

What Has Been Built:

- **Syntax**: How to write statements in FOL.
- **Unification**: How to make expressions match.

Why does matching matter?

→ Because rules are *general* but facts are *specific*!

What is missing?

→ How to *derive* new knowledge!

Goal: Automated Reasoning

Input: Facts and rules in a knowledge base

Output: Answers to queries (true / false or variable bindings)

Process:

1. Find a rule that might apply to the query.
2. Use unification to match the rule's premises with known facts.
3. Apply the same substitution to the rule's conclusion.
4. Derive new knowledge and repeat as needed.

Knowledge Base:

Student(Alice)

$\forall x$ Student(x) $\Rightarrow$ TakesCourses(x)

$\forall x$ TakesCourses(x) $\Rightarrow$ NeedsCoffee(x)

Query: NeedsCoffee(Alice)?

Yes.

We need formal rules for deriving new knowledge, using quantifiers, and making valid substitutions.

Inference Rules: Universal Instantiation

Rule: If something is true for *all* x, then it is true for any *specific* instance.

Formally:

Given: $\forall x\, P(x)$

Infer: $P(c)$ where c is any term

KB:

Student(Alice)
Student(Bob)
Student(FatherOf(Charlie))
∀x Student(x) ⇒ Person(x)

Using universal instantiation, we can infer:

Student(Alice) ⇒ Person(Alice)
Student(Bob) ⇒ Person(Bob)
Student(FatherOf(Charlie)) ⇒ Person(FatherOf(Charlie))

Inference Rules: Existential Instantiation

Rule: If we know something with property P exists, we can introduce a new constant to refer to it.

Formally

Given: $\exists x\, P(x)$

Infer: $P(k)$ where k is a new constant (called Skolem constant): a unique new symbol introduced to represent the specific object whose existence is claimed

---

KB:

$\exists$x Student(x) $\wedge$ Smart(x)

"There exists a student who is smart"

Using existential instantiation, we can infer:

Student(S_1) $\wedge$ Smart(S_1)

"Let S_1 be a Skolem constant that names the smart student."

We don't know ***which*** student is smart, only that at least one exists so we need to use a *new* symbol not used elsewhere in the KB.

Generalized Modus Ponens (GMP)

Universal Instantiation + Unification + Modus Ponens = Generalized Modus Ponens

If we can unify facts with the premises of a rule, we can infer the conclusion with that same substitution.

Suppose we have:

Knowledge Base:

Facts: $f_1, f_2, \ldots, f_n$

Rule(s): $\forall x (r_1, r_2, \ldots, r_n) \Rightarrow s$

And we have a substitution Θ where for all i:

$$f_i = \Theta(r_i)$$

We can infer:

$$\Theta(s)$$

Since s is a sentence that may contain variables that got bound during unification, we infer $\Theta(s)$.

Example of Generalized Modus Ponens

KB:

Teaches(Alvin, CSC343, Spring2026)
Enrolled(Alice, CSC343, Spring2026)

Rule(s):

$\forall$prof $\forall$student $\forall$course $\forall$semester (Teaches(prof, course, semester) $\wedge$ Enrolled(student, course, semester))
$\Rightarrow$ InClass(student, prof, course)

Match premises with facts using Θ:

Θ(Teaches(prof, course, semester)) = Teaches(Alvin, CSC343, Spring2026)

where Θ(prof) = Alvin, Θ(course) = CSC343, and Θ(semester) = Spring2026

Θ(Enrolled(student, course, semester)) = Enrolled(Alice, CSC343, Spring2026))

where Θ(student) = Alice, Θ(course) = CSC343, and Θ(semester) = Spring2026

So by generalized Modus Ponens, we have:

Θ(InClass(student, prof, course)) = InClass(Alice, Alvin, CSC343)

Forward Chaining in FOL

1. Start with initial facts in KB
2. Find a rule whose premises match facts (using unification)
3. Apply GMP to derive the conclusion
4. Add the new fact to KB
5. Repeat until goal is derived or no new facts can be added

This is the same algorithm as in propositional logic, except here we use unification and GMP to derive new facts in FOL.

Properties

Sound: Everything derived is entailed by the KB

- Forward chaining only derives logical consequences
- No false conclusions from true premises

Exhaustive: Derives all entailed facts

- Continues until no new inferences
- Guarantees nothing is missed

Complete for definite clauses: Find all entailed atomic facts

- Definite clause: $(p_1 \wedge p_2 \wedge \cdots \wedge p_n) \Rightarrow q$
  (single positive conclusion)
- If entailed, forward chaining will eventually derive it

May derive irrelevant facts: the "Shotgun approach"

- Produces facts unrelated to your query
- Inefficient for large KBs

Backward Chaining in FOL

1. Check if goal is already a fact in KB
2. Find rules whose conclusions could match the goal
3. Use UNIFY to match the rule's conclusion with the goal
4. If unification succeeds, apply θ to the rule's premises to get specific subgoals
5. Recursively apply backward chaining to prove each subgoal
6. If all subgoals are proven, return success; otherwise try next rule
7. If no rule succeeds, return failure

Same algorithm as in propositional logic, except here we use unification to work backwards from goals to facts in FOL.

Properties

Complete for definite clauses: Finds a proof if one exists

- Definite clause: $(p_1 \wedge p_2 \wedge \cdots \wedge p_n) \Rightarrow q$
- If entailed, backward chaining will prove it

Goal-Directed: More efficient than forward chaining

- Explores only facts relevant to the query
- Avoids irrelevant derivations

Sound: Proves only entailed goals

- No false conclusions
- Every step is logically valid

Prolog-based:

- Prolog uses backward chaining with depth-first search
- Ideal for answering queries

Example: Backward Chaining in FOL

KB:

1. Student(Alice)
2. Takes(Alice, CSC343)
3. ∀x (Student(x) ∧ Takes(x, CSC343)) ⇒ Smart(x)

Query: Smart(Alice)

In KB.

$\text{Student}(\text{Alice})$

In KB.

$\text{Takes}(\text{Alice}, \text{CSC343})$

Prove subgoals

$$\text{Student}(\text{Alice}) \wedge \text{Takes}(\text{Alice}, \text{CSC343})$$

Apply Θ to premises to get subgoals.

$$\Theta = \text{UNIFY}\big(\text{Smart}(x), \text{Smart}(\text{Alice})\big) = \{\, x/\text{Alice} \,\}$$

UNIFY goal with rule conclusion:

∀x (Student(x) ∧ Takes(x, CSC343)) ⇒ Smart(x)

Find matching rule: conclusion could match the goal

Smart(Alice)

Inference in Prolog

Backtracking

If we reach a point where a goal cannot be matched, or the body of a rule cannot be matched, we *backtrack* to the last (most recent) spot where a choice of matching a particular fact or rule was made.

We then try to match a different fact or rule. If this cannot be done, we go back to the next previous place where a choice was made and try a different match there.

We try alternatives until we are able to solve all the goals in our query or until all possible choices have been tried and found to fail.

If this happens, we answer “no” the query can’t be solved.

As we try to match facts and rules we try them in their order of definition.

Depth-first search.

Backtracking Example

Our database:

```
grandMotherOf(X,GM) :- motherOf(X,M), motherOf(M,GM).
grandMotherOf(X,GM) :- fatherOf(X,F), motherOf(F,GM).
fatherOf(tom,dick).
fatherOf(dick,harry).
fatherOf(jane,harry).
motherOf(tom,judy).
motherOf(dick,mary).
motherOf(jane,mary).
```

Query:

```
?- grandMotherOf(tom, GM).
```

We try the first `grandMotherOf` rule first.

This forces `X = tom`.

We have to solve:

```
motherOf(tom,M), motherOf(M,GM).
```

This forces `M = judy`.

⊗

No other `motherOf` rule can solve `motherOf(tom,M)`.

```
motherOf(judy,GM)
```

⊗

None of the `motherOf` rules match this goal, so we backtrack.

Lists in Prolog

The notation [H | T] represents a list with H matching the head of the list and T matching the rest of the list.

```
[1, 2, 3] = [1 | [2,3]] = [1, 2 | [3]] = [1, 2, 3 | [] ]
```

```
?- [1, 2, 3] = [1 | [2,3]], [1, 2 | [3]] = [1, 2, 3 | [] ].
true.
```

"_" (underscore) can be used as a wildcard or "don't care" symbol in matches.

```
?- [user].
|: p([1,2,3,4]).
```

```
?- p([X|Y]).
X = 1,
Y = [2, 3, 4].
```

```
?- p([_, _, X | Y]).
X = 3,
Y = [4].
```

```
?- p([H, _ | T]).
H = 1,
T = [3, 4].
```

List Operations

List operations are defined using rules and facts.

The definitions are similar to those used in Scheme or ML, but they are *non-procedural*. That is, you do not give an execution order.

Instead, you state recursive rules and non-recursive "base cases" that characterize the operation you are defining.

```
append([], L, L).          % Base Case
```

input output

An empty list (argument 1) appended to any list L (argument 2) gives L (argument 3) as its answer.

```
append([H | T1], L2, [H | T3]) :- append(T1, L2, T3).    % Recursion
```

input output

If we take a list that begins with H and has T1 as the rest of the list and append it to a list L2 then the resulting appended list will begin with H.

The rest of the resulting list, T3, is the result of appending T1 (the rest of the first list) with L2 (the second input list).

Example: append

```
append([], L, L).
append([H | T1], L2, [H | T3]) :- append(T1, L2, T3).
```

Unify([], T1=[]) SUCCESS

Unify(L, L2=[2,3]) SUCCESS

Unify(L=[2,3], T3) SUCCESS

append([], L, L) SUCCESS

T3=[2,3]

Unify(H, 1) SUCCESS

Unify(T1, []) SUCCESS

Unify(L2, [2,3]) SUCCESS

Unify(T, [1 | T3]) SUCCESS

Unify([], [1]) FAILS

append([], L, L)

append([H | T1], L2, [H | T3]) :- append(T1, L2, T3). SUCCESS

T = [1,2,3] SUCCESS

append([1], [2,3], T)

SUCCESS

Ancestral Backtracking Example (continued)

```
grandMotherOf(X,GM) :- motherOf(X,M), motherOf(M,GM).
grandMotherOf(X,GM) :- fatherOf(X,F), motherOf(F,GM).
fatherOf(tom,dick).
fatherOf(dick,harry).
fatherOf(jane,harry).
motherOf(tom,judy).
motherOf(dick,mary).
motherOf(jane,mary).
```

Query: `?- grandMotherOf(tom, GM).`

We try the second `grandMotherOf` rule.

This forces `X = tom`.

We have to solve:

`fatherOf(tom,F), motherOf(F,GM).`

We can match:

`fatherOf(tom,dick).`

This forces `F = dick`.

⊗

None of the `fatherOf` rules match this goal, so we backtrack.

`motherOf(dick,GM)`

We can match:

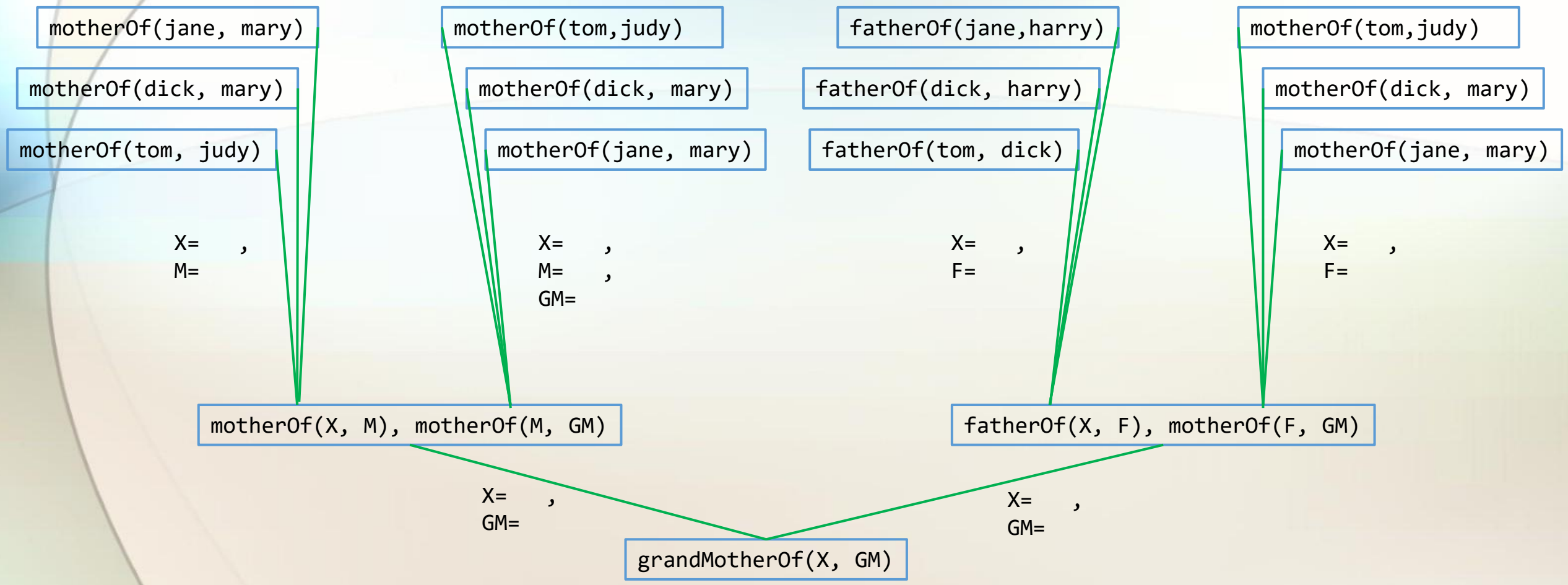
`motherOf(dick, mary).`

We have matched all our goals, so we know the query is true, with `GM = mary`. ☺

⊗

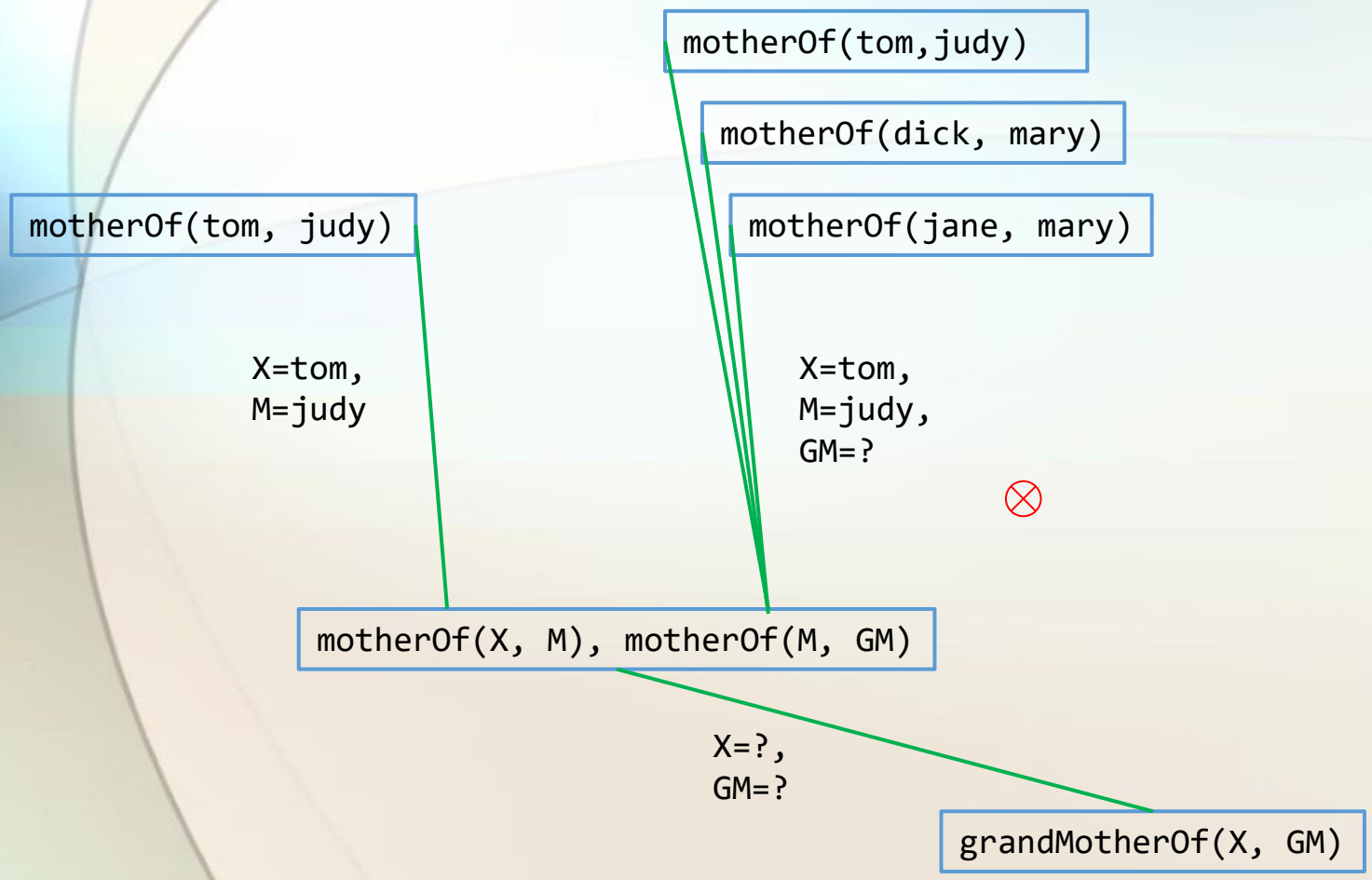
No other `motherOf` rule can solve `motherOf(dick, GM)`.

Query Overview



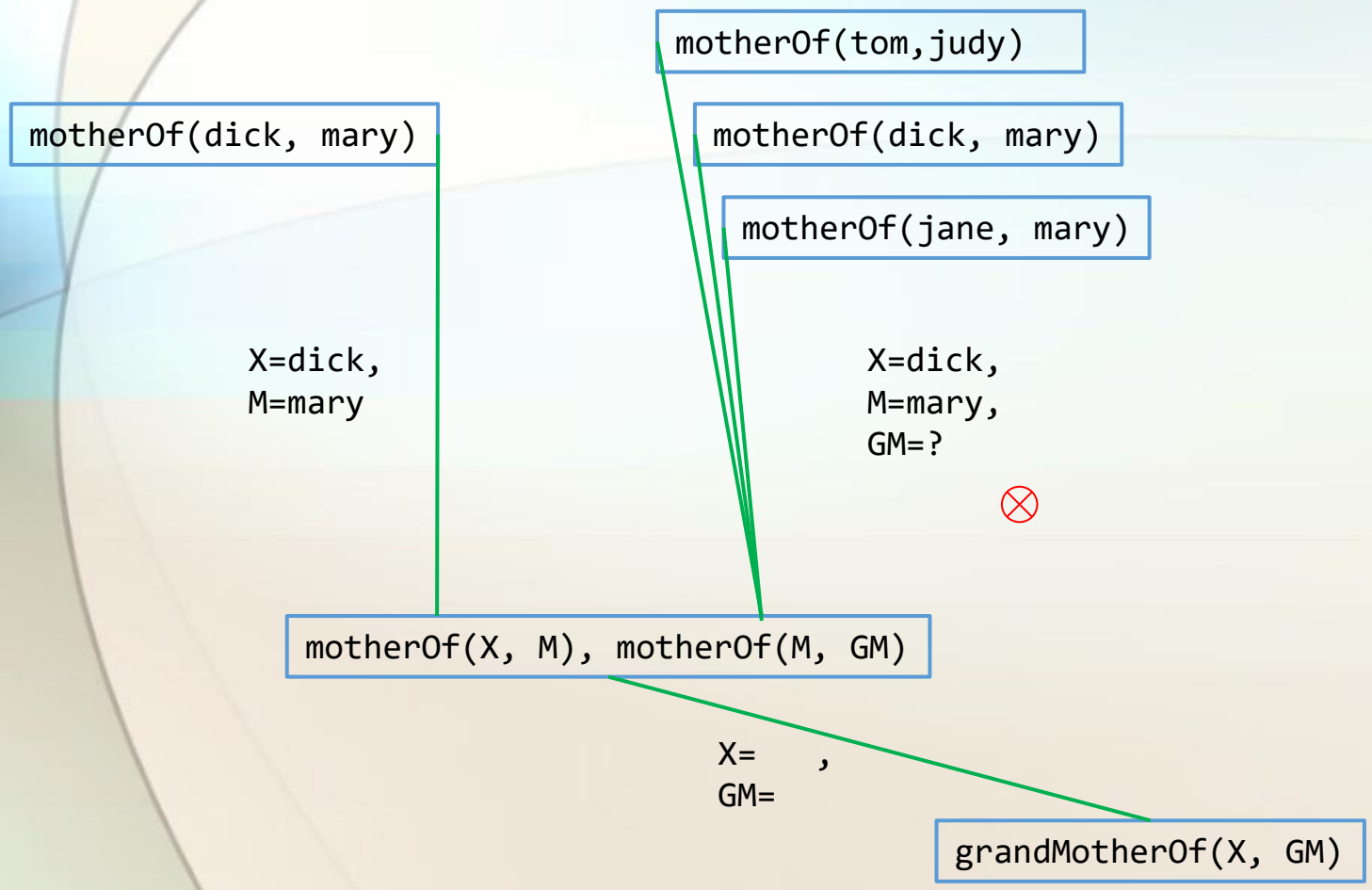
Visualizing Prolog Query Search I

```
grandMotherOf(X,GM) :-
    motherOf(X,M), motherOf(M,GM).
grandMotherOf(X,GM) :-
    fatherOf(X,F), motherOf(F,GM).
fatherOf(tom,dick).
fatherOf(dick,harry).
fatherOf(jane,harry).
motherOf(tom,judy).
motherOf(dick,mary).
motherOf(jane,mary).
```



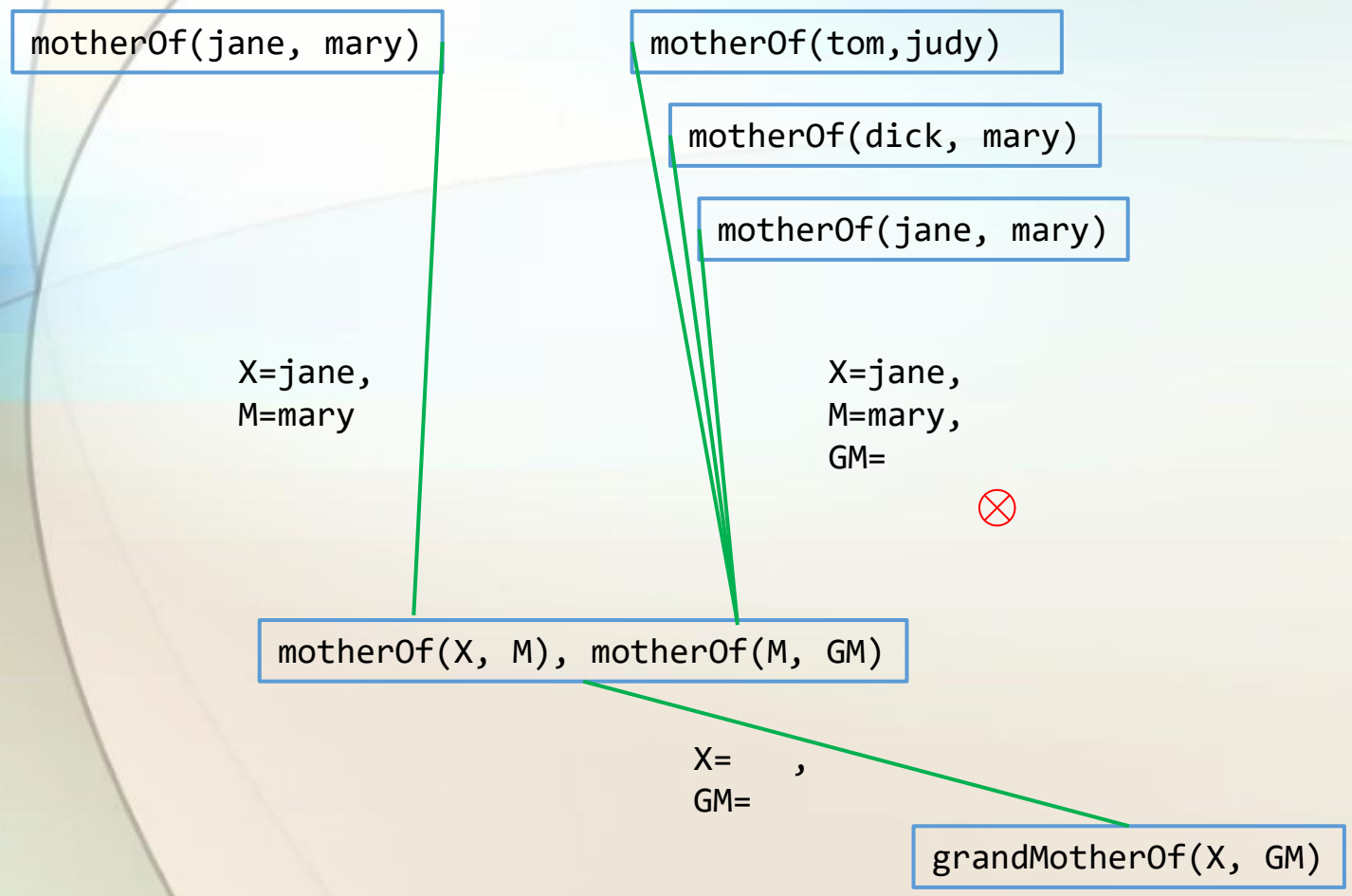
Visualizing Prolog Query Search II

```
grandMotherOf(X,GM) :-
    motherOf(X,M), motherOf(M,GM).
grandMotherOf(X,GM) :-
    fatherOf(X,F), motherOf(F,GM).
fatherOf(tom,dick).
fatherOf(dick,harry).
fatherOf(jane,harry).
motherOf(tom,judy).
motherOf(dick,mary).
motherOf(jane,mary).
```

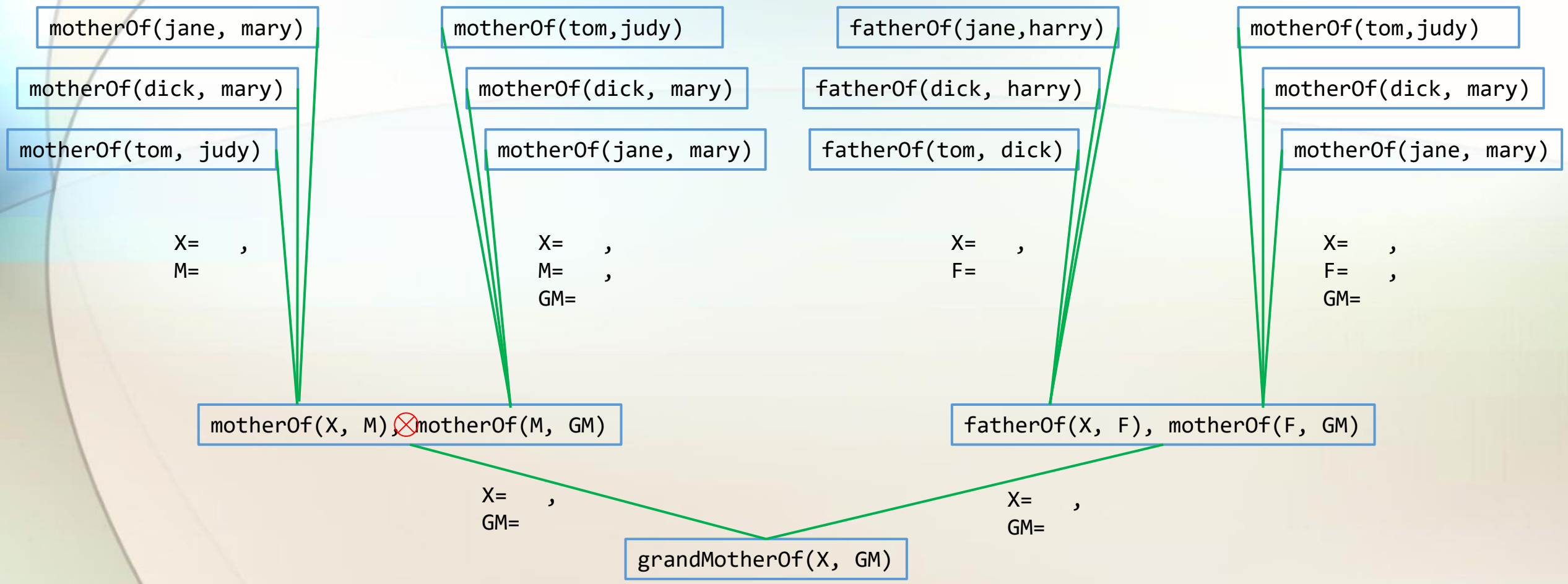


Visualizing Prolog Query Search III

```
grandMotherOf(X,GM) :-
    motherOf(X,M), motherOf(M,GM).
grandMotherOf(X,GM) :-
    fatherOf(X,F), motherOf(F,GM).
fatherOf(tom,dick).
fatherOf(dick,harry).
fatherOf(jane,harry).
motherOf(tom,judy).
motherOf(dick,mary).
motherOf(jane,mary).
```

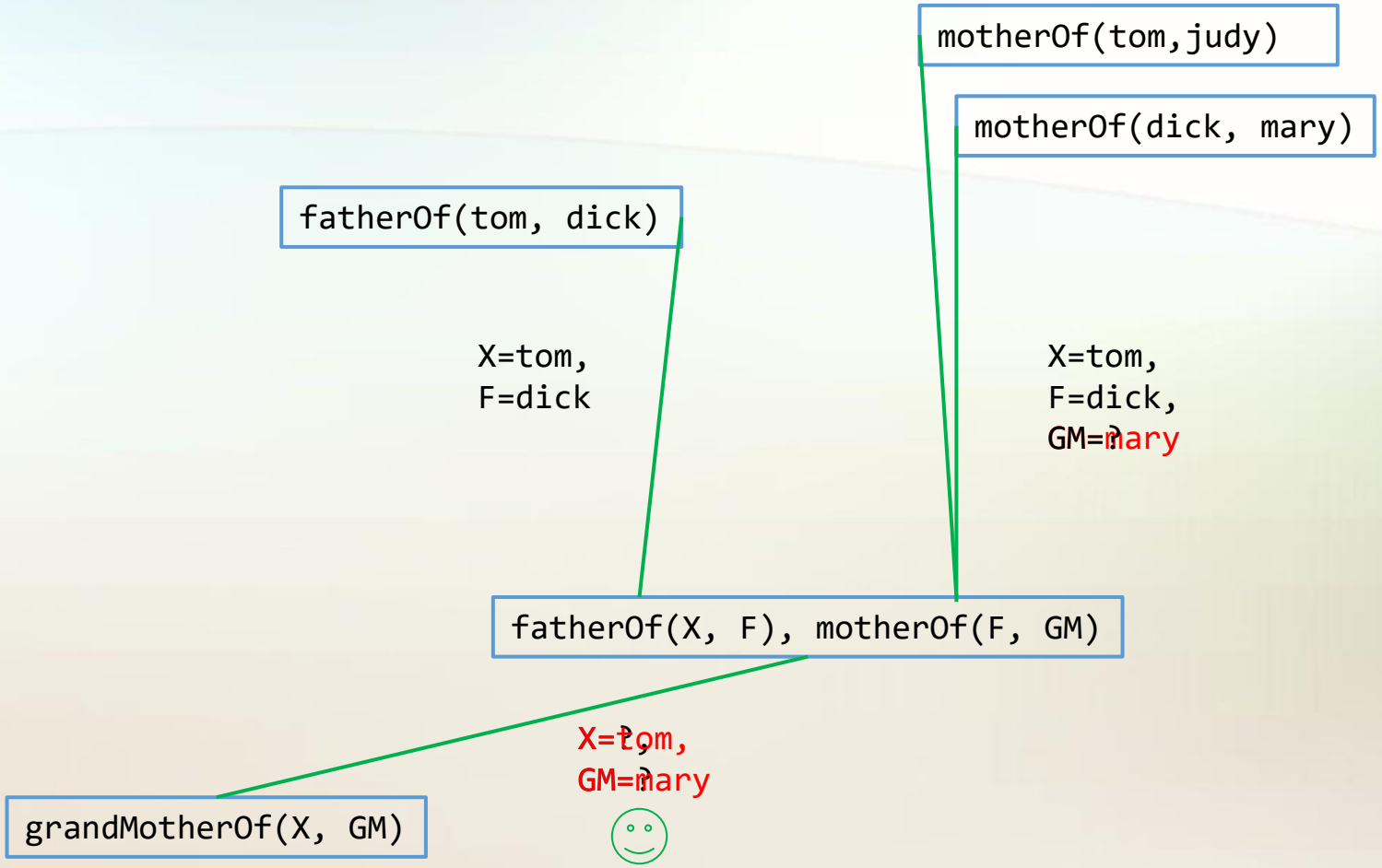


Visualizing Prolog Query Search IV: New Rule



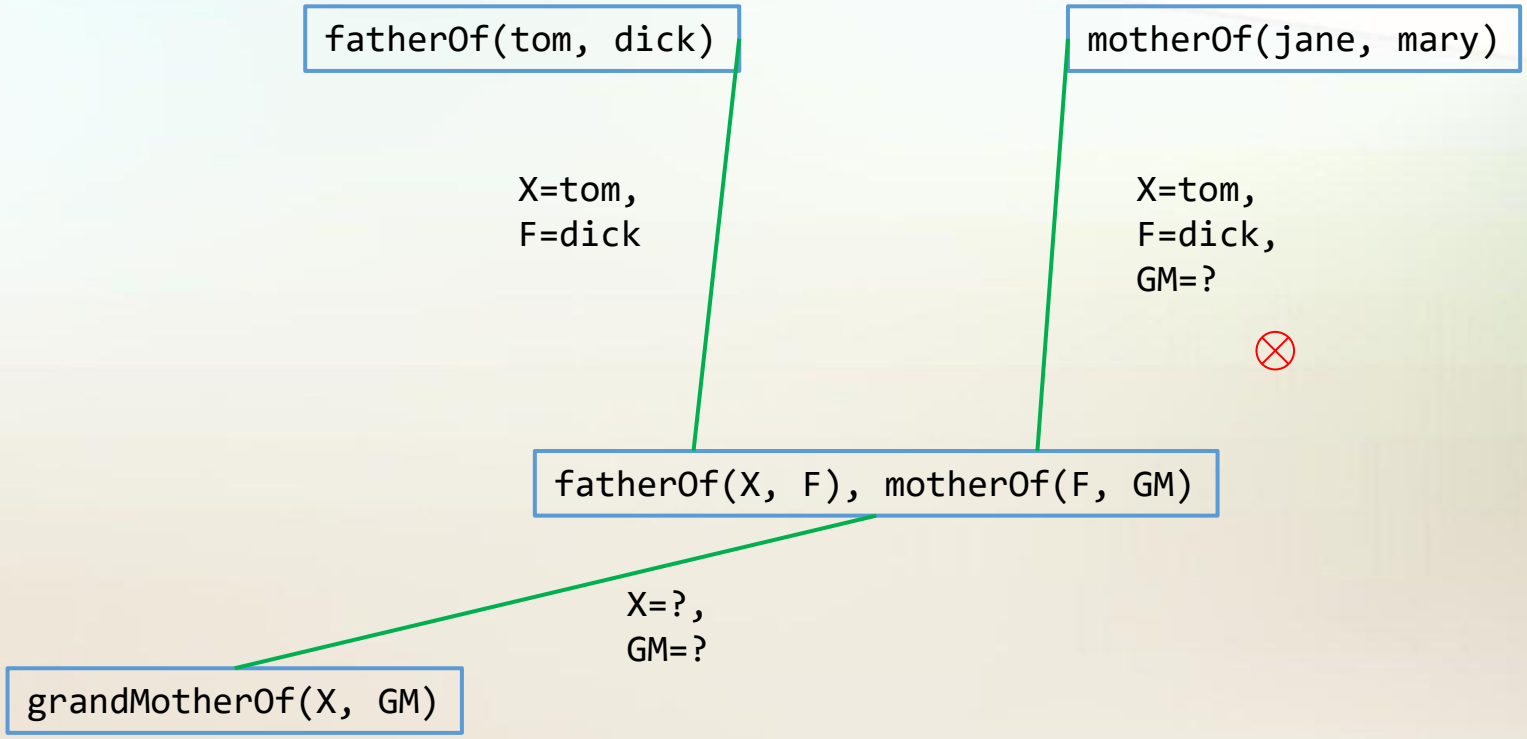
Visualizing Prolog Query Search V

```
grandMotherOf(X,GM) :-
    motherOf(X,M), motherOf(M,GM).
grandMotherOf(X,GM) :-
    fatherOf(X,F), motherOf(F,GM).
fatherOf(tom,dick).
fatherOf(dick,harry).
fatherOf(jane,harry).
motherOf(tom,judy).
motherOf(dick,mary).
motherOf(jane,mary).
```



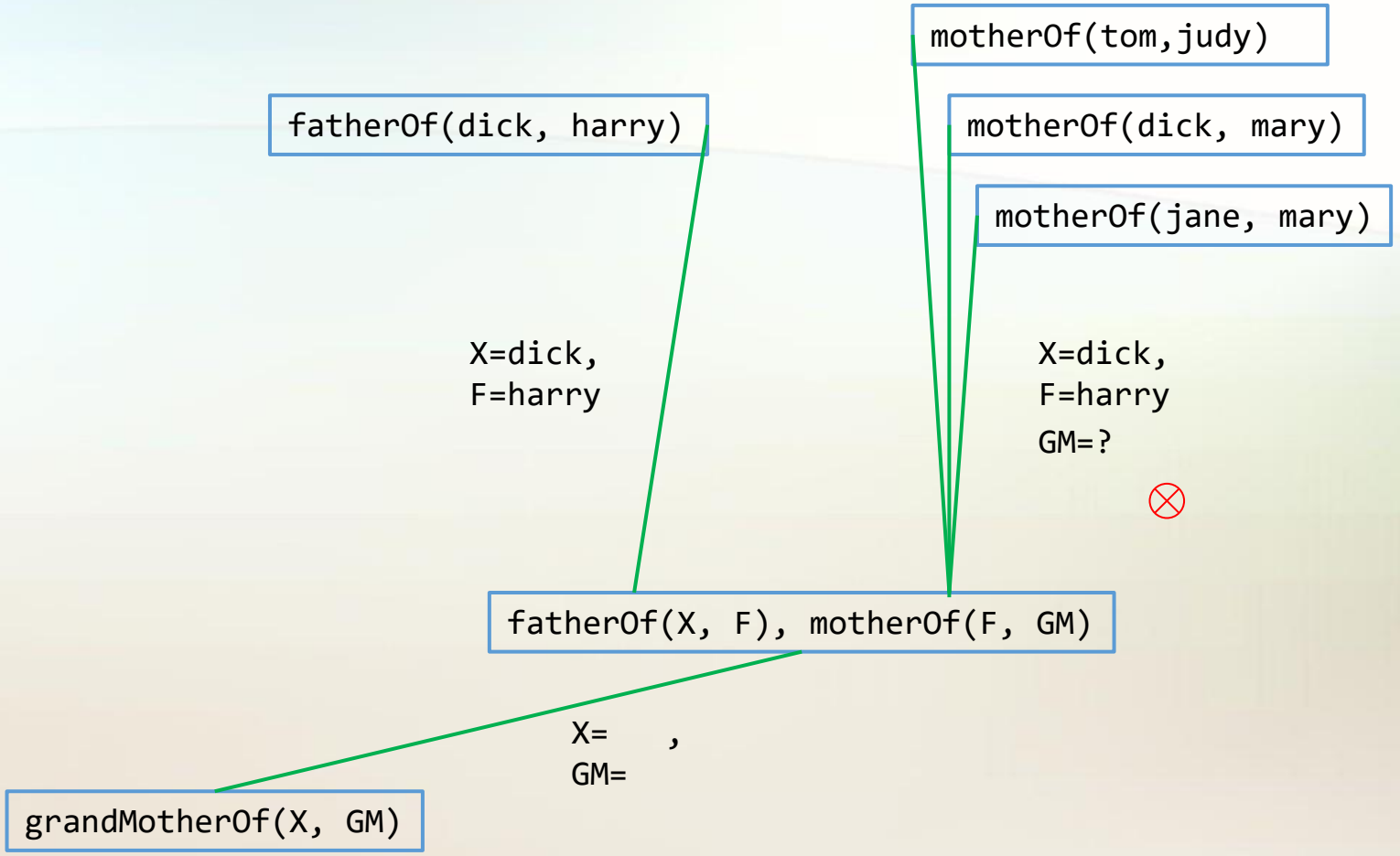
Visualizing Prolog Query Search VI

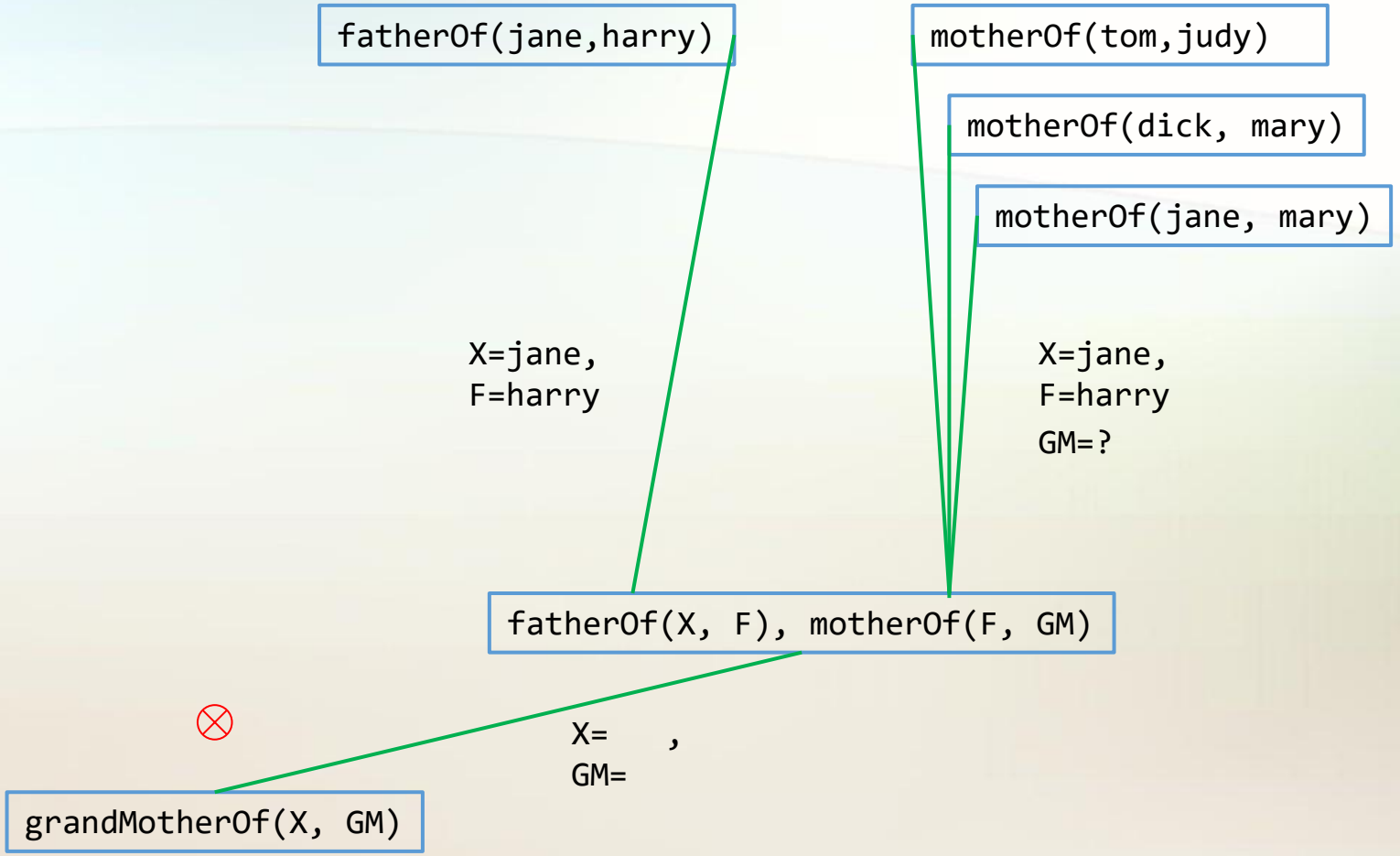
```
grandMotherOf(X,GM) :-
    motherOf(X,M), motherOf(M,GM).
grandMotherOf(X,GM) :-
    fatherOf(X,F), motherOf(F,GM).
fatherOf(tom,dick).
fatherOf(dick,harry).
fatherOf(jane,harry).
motherOf(tom,judy).
motherOf(dick,mary).
motherOf(jane,mary).
```



Visualizing Prolog Query Search VII

```
grandMotherOf(X,GM) :-
    motherOf(X,M), motherOf(M,GM).
grandMotherOf(X,GM) :-
    fatherOf(X,F), motherOf(F,GM).
fatherOf(tom,dick).
fatherOf(dick,harry).
fatherOf(jane,harry).
motherOf(tom,judy).
motherOf(dick,mary).
motherOf(jane,mary).
```



Visualizing Prolog Query Search VIII

```
grandMotherOf(X,GM) :-
    motherOf(X,M), motherOf(M,GM).
grandMotherOf(X,GM) :-
    fatherOf(X,F), motherOf(F,GM).
fatherOf(tom,dick).
fatherOf(dick,harry).
fatherOf(jane,harry).
motherOf(tom,judy).
motherOf(dick,mary).
motherOf(jane,mary).
```